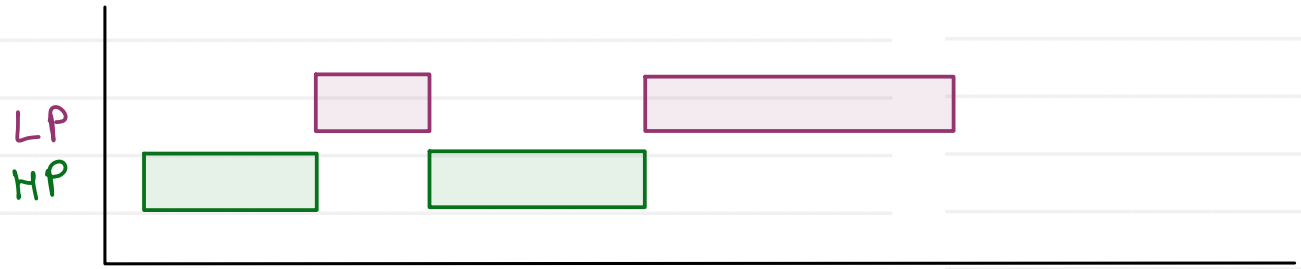
Sunbeam Institute of Information Technology
Pune and Karad

Embedded and RTOS (FreeRTOS) Training At RIT College, Islampur

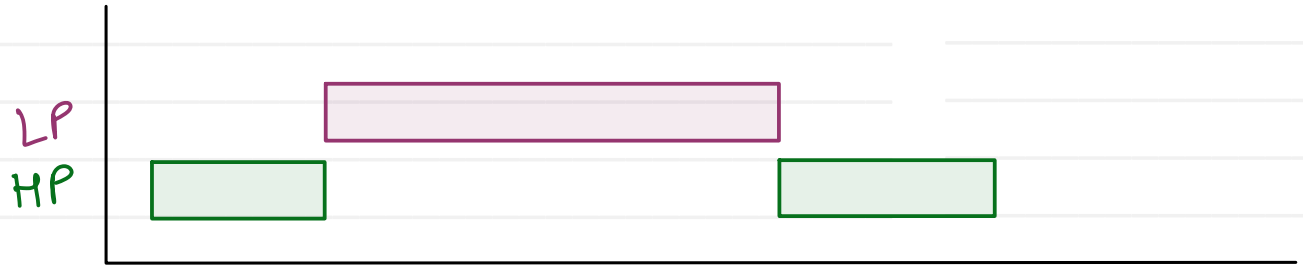
Trainer - Devendra Dhande

Email – devendra.dhande@sunbeaminfo.com

configUSEPreemption - Enabled ← pre emptive scheduling



configUSEPreemption - Disabled ← cooperative scheduling



FreeRTOS - Queue

- **What is a Queue?**
  - Queue is an Inter-Task Communication (ITC) mechanism.
  - Used to transfer data between tasks.
  - Follows FIFO (First In First Out).
  - Supports synchronization and data exchange.

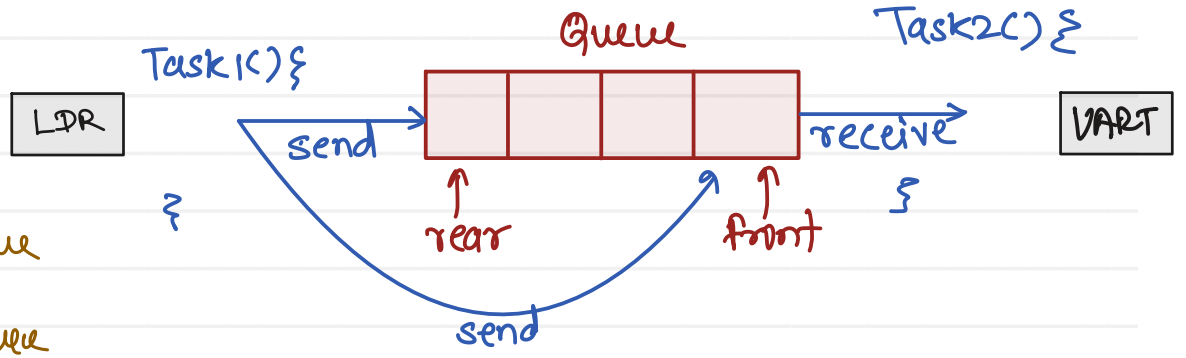
- **Why Queue is Required?**
  - Safe data sharing between tasks.
  - Avoids global variables.
  - Supports blocking operations.
  - Provides task synchronization.

sender will block, if writes into full queue

receiver will block, if reads from empty queue

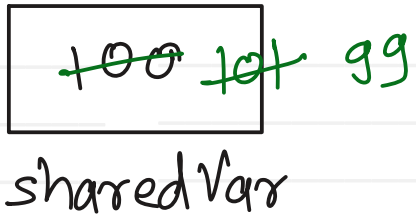
- **Queue Characteristics**
  - FIFO data structure.
  - Stores fixed-size items.
  - Multiple producers and consumers supported.
  - Thread-safe.

- **Queue Operations**
  - Create Queue - xQueueCreate()
  - Send Data - xQueueSend() / xQueueSendFromISR()
  - Receive Data - xQueueReceive()
  - Delete Queue - vQueueDelete()
- **Queue Working**



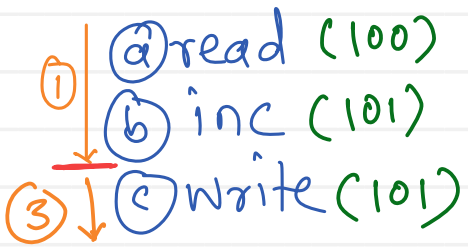
- **Applications**
  - Sensor Data Transfer
  - UART Communication
  - Command Processing
  - Event Handling
- **Advantages**
  - Safe communication
  - Data buffering
  - Synchronization
  - RTOS-aware

Race condition



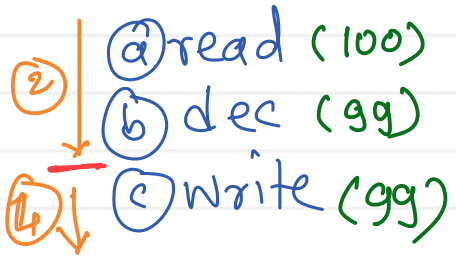
Task1( ) {

sharedVar++;



}

Task2( ) {



sharedVar--;

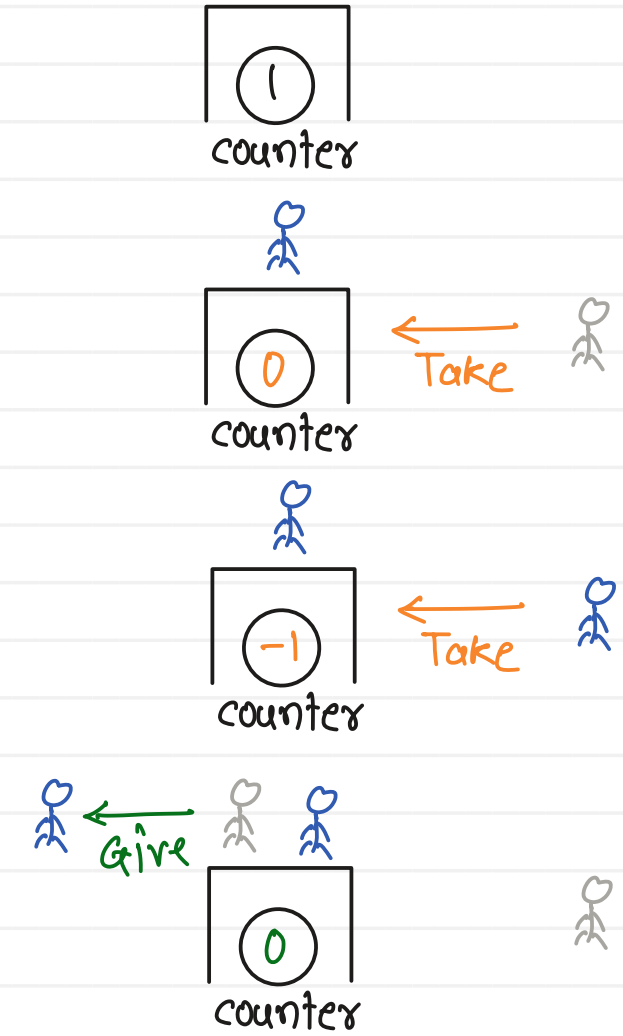
}

Race condition / Data inconsistancy / Peterson's problem:

- when global/shared variables/resources are used by multiple tasks simultaneously, it leads to corruption of the resource/variable
- solution to this problem is synchronization

Semaphore

- internally every semaphore is a counter
- Operations
  1. Dec / wait / Take :
     - dec counter
     - if counter < 0, then block current or calling task
  2. Inc / Post / Give :
     - inc counter
     - if few tasks are blocked on the semaphore, then wakeup one



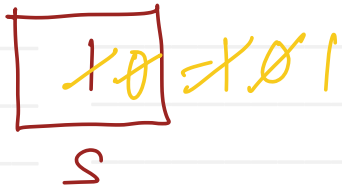
Mutual Exclusion


```
Task1( ){
    Take();
    sharedVar++;
    Give()
}
```

1. Take(s)
   a. read (100)
   b. inc (101)
   c. write (101)
3. Give(s)

2. Take(s)
4. a. read (101)
   b. dec (100)
   c. write (100)
   Give(s)

```
Task2( ){
    Take();
    sharedVar--;
    Give();
}
```



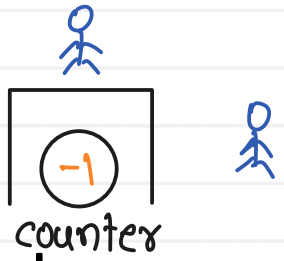
Mutual Exclusion :
- only one will use a resource at a time
- this is achieved with the help of either binary semaphore or mutex

Semaphore

- **What is a Semaphore?**
  - Synchronization mechanism in FreeRTOS.
  - Used for signaling between tasks and ISRs.
  - Does not transfer data.
  - Controls access to resources/events.

- **Why Semaphore is Required?**
  - Task synchronization.
  - ISR-to-Task communication.
  - Event signaling.
  - Resource availability indication.

- **Types of Semaphores**
  - **Binary Semaphore**
    - Used for:
      - Event signaling
      - ISR synchronization



  - **Counting Semaphore**
    - Used for:
      - Event counting
      - Resource counting

- **Applications**
  - Button Interrupt Handling
  - UART Event Notification
  - ADC Conversion Complete
  - DMA Transfer Complete

- **Advantages**
  - Fast synchronization
  - Low memory overhead
  - ISR compatible

Mutex

- **What is a Mutex?**
  - Mutex = Mutual Exclusion
  - Protects shared resources
  - Only one task can own a mutex at a time
  - Prevents race conditions
  - Supports Priority Inheritance

- **Applications**
  - UART Protection
  - SPI Protection
  - I2C Protection
  - LCD Protection
  - Shared Buffer Protection

- **Advantages**
  - Prevents Race Conditions
  - Resource Ownership
  - Priority Inheritance
  - Easy Resource Protection

- **Mutex Operations**
  - Create - xSemaphoreCreateMutex()
  - Lock - xSemaphoreTake()
  - Unlock - xSemaphoreGive()

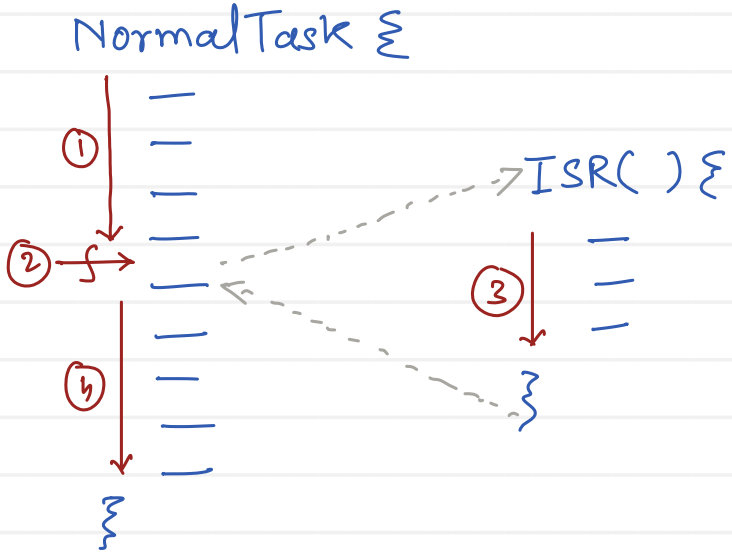
Differenace Between Mutex and Semaphore

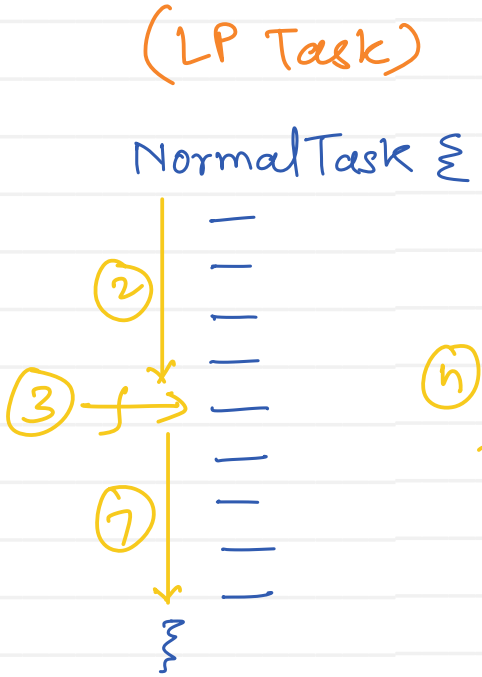
Mutex	Semaphore
Resource Protection	Event Signaling
Ownership	No Ownership
Priority Inheritance	No Priority Inheritance
ISR Not Allowed	ISR Allowed

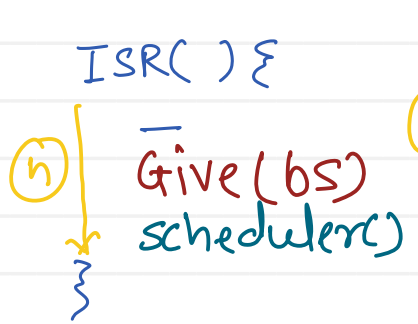
FreeRTOS - Deferred Processing

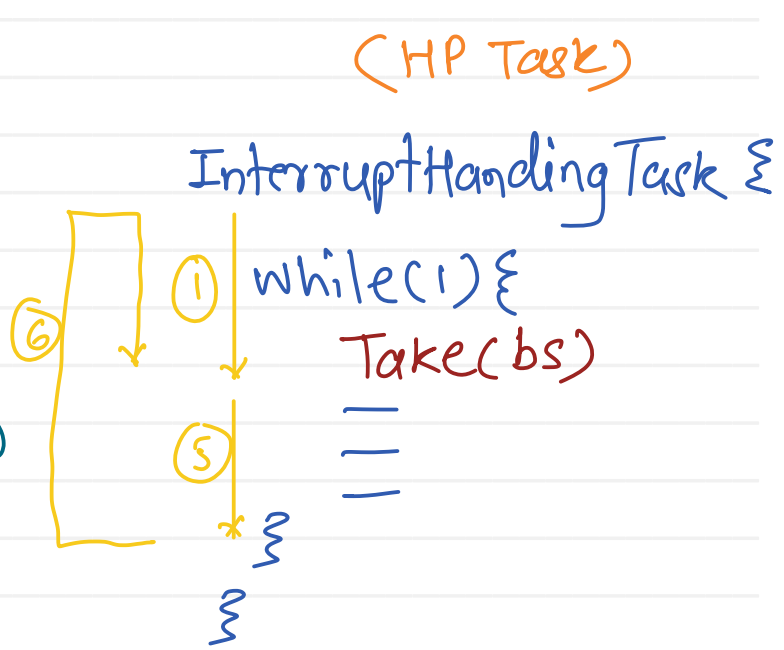
- ISR should be short and fast.
- Heavy processing should not be done inside ISR.
- ISR only captures event information.
- Actual processing is deferred to a task.
- Reduces interrupt latency.
- Improves system responsiveness.

- **Advantages**
  - Fast ISR Execution
  - Reduced Interrupt Latency
  - Better Real-Time Performance
  - Easier Debugging









FreeRTOS - Task Notifications

- **What are Task Notifications?**
  - Lightweight communication mechanism in FreeRTOS.
  - Used for task synchronization and event signaling.
  - Faster and more memory-efficient than semaphores.
  - Each task has a built-in notification value.
- **Why Use Task Notifications?**
  - Very low overhead.
  - Fastest ISR-to-Task communication.
  - No separate object creation required.
  - Suitable for simple event signaling.
- **Applications**
  - Button Interrupt Handling
  - UART Event Notification
  - ADC Conversion Complete
  - DMA Transfer Complete
- **Advantages**
  - Fastest ITC mechanism
  - Low RAM usage
  - ISR compatible
  - Easy to use
- **Common APIs**
  - **Send Notification**
    - xTaskNotify()
    - xTaskNotifyGive()
  - **Wait for Notification**
    - xTaskNotifyWait()
    - ulTaskNotifyTake()

FreeRTOS – Gate Keeper Task

- **What is Gatekeeper Task?**
  - Dedicated task that owns a shared resource.
  - Other tasks never access the resource directly.
  - Requests are sent to Gatekeeper Task.
  - Gatekeeper performs all resource operations.

- **Advantages**
  - No Mutex Required
  - Eliminates Race Conditions
  - Centralized Resource Access
  - Easier Resource Management